

B4-Browse Feature Design & AI Implementation Prompt

Diagnostic Module: Hanging Services & Ghost Kernel Drivers Detection

Project Target	B4-Browse (C# System Diagnostic / Safe Browser Architecture Auditor)
Feature Objective	Detect and report hidden driver/service blocks (e.g., <code>STOP_PENDING</code> , <code>DELETE_PENDING</code>)
Document Purpose	Reusable specifications and plug-and-play AI prompt to generate the implementation code.

1. Background & Architectural Context

When diagnosing system integrity, web browser stability, or installation rollbacks, standard utilities like `services.msc` fail to report critical operational anomalies: **Hanging Kernel Drivers** and **Ghost Services** trapped in transient states.

For instance, network filtering drivers (like NetLimiter's `nldrv`) can become jammed in a `STOP_PENDING` state. Because they are flagged with `IGNORES_SHUTDOWN` , Windows Hybrid Boot/Fast Startup preserves this broken state across typical system reboots, generating fatal Win32 errors such as `ERROR_SERVICE_MARKED_FOR_DELETION` (`0x430 / 1072`) during updates or installs.

Integrating a dedicated scanner into **B4-Browse** exposes these root-cause infrastructure issues automatically.

2. Technical Specifications

- **Target States:** Enumerate and specifically flag entries where `dwCurrentState == SERVICE_STOP_PENDING (0x00000003)` or `SERVICE_START_PENDING (0x00000002)` .
- **Subsystem Focus:** Distinctly isolate `SERVICE_KERNEL_DRIVER (0x00000001)` and `SERVICE_FILE_SYSTEM_DRIVER (0x00000002)` instances, as network hooks directly impact the diagnostic stack.
- **Implementation Strategy:** Avoid shell execution overhead (e.g., `sc query` parsing). Utilize direct P/Invoke calls to `Advapi32.dll` or WMI namespaces via `ManagementObjectSearcher` for robust, performance-conscious execution.

AI Execution Prompt (Copy/Paste Section)

Provide the text block below to an AI model to automatically generate the complete module for B4-Browse:

```
Act as an expert C# systems engineer. Write a robust diagnostic module for the C# application "B4-Browse" that scans the Windows operating system for "Ghost Services" and "Hanging Kernel Drivers" that are trapped in transient states (such as STOP_PENDING or START_PENDING).
```

Requirements:

1. Architectural Target: Integrate cleanly into a standard C# environment (.NET Core / .NET Framework).
2. Detection Scope: Discover all system services and device drivers where the current execution state is stuck or transitioning (specifically `ServiceState.StopPending` and `ServiceState.StartPending`).
3. Underlying Mechanisms: Provide two implementation strategies within the code for flexibility:
 - Method A (Preferred/Performant): Native Win32 API interactions using P/Invoke mapping over `Advapi32.dll` (utilizing `OpenSCManager`, `EnumServicesStatusEx`, and `SERVICE_STATUS_PROCESS`).
 - Method B (Fallback): A lightweight WMI query targeting the `Win32_Service` infrastructure.
4. Output Schema: Return a structured collection of strongly-typed objects containing:
 - Service/Driver System Name
 - Display Name
 - Service Type (Kernel Driver vs. User Service)
 - Current Trapped State (e.g., `StopPending`)
 - A contextual diagnostic recommendation (e.g., indicating if a cold restart via `shutdown /r /t 0` or a explicit registry key purge under `HKLM\SYSTEM\CurrentControlSet\Services\` is necessary to clear the lock).
5. Error Resilience: Wrap all logic in strict try-catch handlers to elegantly handle elevation/Access Denied anomalies without crashing the broader B4-Browse suite.

Provide clean, well-commented, production-ready code.

3. Reference Implementation Blueprint (Win32 P/Invoke Structure)

To assist the engine parsing the prompt, ensure the foundational win32 structures map correctly to these definitions:

```
[StructLayout(LayoutKind.Sequential, CharSet = CharSet.Unicode)]
public struct ENUM_SERVICE_STATUS_PROCESS
{
```

```
    public string lpServiceName;
    public string lpDisplayName;
    public SERVICE_STATUS_PROCESS ServiceStatusProcess;
}

[StructLayout(LayoutKind.Sequential)]
public struct SERVICE_STATUS_PROCESS
{
    public int dwServiceType;
    public int dwCurrentState;
    public int dwControlsAccepted;
    public int dwWin32ExitCode;
    public int dwServiceSpecificExitCode;
    public int dwCheckPoint;
    public int dwWaitHint;
    public int dwProcessId;
    public int dwServiceFlags;
}
```